what to look for to be able to lift the ceiling. Otherwise over time, you'll wind up with multiple gems in the Gemfile that you want to upgrade but are afraid of negative side effects. Ask us how we know.

You can use the full set of Ruby operators for version specifiers—this table outlines what they mean in context.

Operator	Description
=	Exact version match. Major, minor, and patch level must be identical.
!=	Any version that is not the one specified.
>	Any version that is greater (even at the patch level) than the one specified.
<	Any version that is less than the one specified.
>=	Any version greater than or equal to the specified version.
<=	Any version less than or equal to the specified version.
~>	Pessimistic version constraint operator, officially. Somtimes called the "twiddle-wakka" or "squiggle rocket" because the Ruby community contains multitudes. This operator allows updated versions if the new version differs from the current version only in the last digit specified.

Table 5—Version operators

That last one (~>) could use a little explanation. The idea is to allow for patch updates that are probably bug fixes but not to allow larger updates that might have breaking changes. So, you might want to specify `gem "rspec", "~> 3.11.0"`, and when RSpec releases a patch version 3.11.1 you will get that version with a `bundle update`, but when they update the minor version to 3.12.0, you won't get that version. The ~> operator is usually used with major, minor, and patch versions specified, but you don't have to do that, you could specify `gem "rspec", "~> 3.11"`, in which case you'd get 3.11.1 and 3.12.0, but not 4.0.0.

There's one other twist here, which is that sometimes you want to use a gem that has been released in a beta version. Gem versioning assumes that a version number is a pre-release version if it contains a letter character. You can get pre-release versions of gems by including letters in your version specifier: `gem "rspec", ">= 3.11.0.beta.1"`.

If you've specified a pre-release version, Bundler will look at all pre-release versions when resolving its install or update, so `>= 3.11.0.beta.1` would still receive the updated version when 3.11.0.beta.2 is released because the operator is greater than and beta.2 is higher than beta.1.

The sorting is alphabetical for sections that contain letters and numeric for sections that don't. Sometimes this can be confusing. RubyGems considers, say 7.0.0.pre.1 to be greater than 7.0.0.beta.1 but Ruby on Rails (to pick one example) releases pre before beta so you can get the wrong version if you are not careful. (Be sure to check the lock file to see what version you are actually using.)

Sometimes you need to use a gem that is so far into pre-release that it hasn't actually been released as a gem and only exists in a git repo or even just in a local directory on your computer. Often you need this to test gems that you are currently developing, or to help test other people's work before official release.

You can get access to unreleased gems via Bundler with different options in the Gemfile. Generically, I can access any remote git server with the `git:` option, as in: